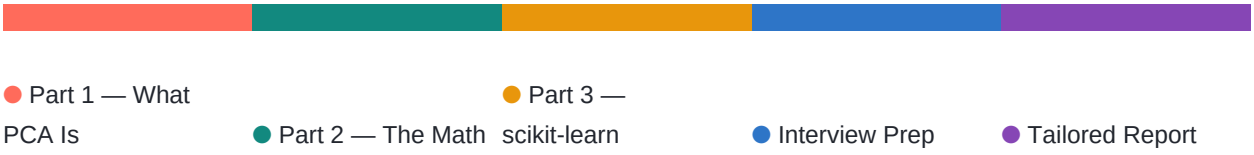
Principal Component Analysis (PCA)

A Complete Study Guide

Andrew Ng · Machine Learning Specialization · Course 3 (Optional Videos)



This guide reconstructs three consecutive optional lecture videos on PCA in the order the instructor presents them: (1) what PCA is and why it's used, (2) the math of how PCA actually picks its axes, and (3) how to run PCA in scikit-learn. The teaching rhythm of each video (intuition first, formal idea second, code/worked numbers last) is preserved rather than reshuffled into a generic outline. An interview-prep section and a tailored takeaway report follow the core content.

Part 1 — What PCA Is and Why We Need It

This video opens right after the recommender-system videos, introducing PCA as the next unsupervised learning algorithm: a tool for taking data with a lot of features and shrinking it down to two or three numbers so it can actually be plotted and looked at.

The motivating problem

You cannot plot 1,000-dimensional data. If a dataset of passenger cars has 50 features (length, width, wheel diameter, height, and so on), there is no direct way to "see" it. PCA's job is to compress a large number of features down to two or three new features so a data scientist can plot the result and visually understand what's going on — including spotting when something unexpected is happening in the data.

Building intuition with cars: three worked scenarios

The instructor builds up intuition using the same two-feature car dataset across three increasingly complex versions, letting the reader see PCA's decision-making before any formula is introduced.

EXAMPLE 1

Case A — length (x_1) vs. width (x_2). Road-width constraints mean car width barely varies (~1.8m for most US cars), while length varies a lot. PCA applied here will, in effect, just keep x_1 and drop x_2, since x_2 carries almost no information.

EXAMPLE 2

Case B — length (x_1) vs. wheel diameter (x_2). Wheel diameter varies a little more than width did, but the pattern is the same: PCA still ends up essentially just keeping x_1.

EXAMPLE 3

Case C — length (x_1) vs. height (x_2). Now both features vary meaningfully — bigger cars tend to be both longer AND taller. Picking only x_1 (ignoring height) or only x_2 (ignoring length) throws away real information. Neither original axis alone is a good summary anymore.

Introducing the z-axis

For Case C, the instructor introduces a new axis, called the z-axis, that is a combination of x_1 and x_2 — not a third physical dimension sticking out of the page, but a new direction drawn flat within the same x_1–x_2 plot. A car's coordinate along this z-axis roughly captures its overall "size," blending length and height into a single number. This is the core idea of PCA: find one or more new axes so that a data point's coordinate(s) on the new axis (axes) still carry the useful information that used to require two (or many) numbers.

Scaling up: 3D and 50D examples

A 3D visualization is used to show that data which technically has 3 features (x_1, x_2, x_3) sometimes lies almost entirely on a thin 2D "pancake" surface within that 3D space. PCA can then reduce x_1, x_2, x_3 down to just Z_1 and Z_2 without losing much, since the third true dimension carried little independent information.

EXAMPLE 4

Countries example (50 features → 2 features). Given ~50 features per country (GDP, per-capita GDP, Human Development Index, life expectancy, etc.), PCA compresses this down to Z_1 and Z_2. In practice Z_1 often loosely tracks overall country size / total GDP, and Z_2 loosely tracks per-person economic activity. On the resulting plot, a large, high-per-capita-GDP country like the United States sits up and to the right; a small but high-per-capita country like Singapore sits lower on Z_1 but still high on Z_2. This is a recurring theme: the new PCA axes are not chosen by hand — the algorithm finds them — but afterward a human can often assign a rough real-world interpretation to what each axis represents.

Closing framing for this video: PCA is one of the most useful first things to run on any new dataset, purely to visualize it and build intuition before deciding what else to do with it. The next video moves from this intuitive picture to the actual mechanics of how PCA chooses its axes.

Part 2 — How PCA Works (the Math)

Setup and required preprocessing

Starting point: a dataset with features x_1 and x_2 (no label y — this is unsupervised). Before anything else, PCA requires the features to be mean-normalized (zero mean, i.e. subtract each feature's mean). If the features are on very different scales — the instructor reuses the housing example, where x_1 = size in square feet (could be in the thousands) and x_2 = number of bedrooms (a small integer) — feature scaling should also be applied before the remaining PCA steps.

Picking a good axis: projection and spread

With the axes temporarily removed from the plot, only five example points and the origin remain. The task is to pick a single new axis (the z-axis) to represent all five points with just one number each. This is done by *projecting* each point onto the candidate axis: drawing a line segment from the point to the axis that meets the axis at a 90-degree (perpendicular) angle, and using the coordinate where it lands.

Three candidate axes are compared side by side:

- **Axis choice 1** (happens to coincide with the original x_1 axis in this example): a decent choice — the five projected points stay fairly spread out, so a lot of the original variation is preserved.
- **Axis choice 2:** a poor choice — the projected points get squished close together, meaning most of the differences between examples are lost.
- **Axis choice 3:** the best of the three — the projected points end up the most spread apart of all the options tried, preserving the most information.

KEY IDEA — FIRST PRINCIPAL COMPONENT

The axis onto which projecting the data produces the largest possible variance (the most spread-out projections). This is exactly the axis PCA will select when asked to reduce the data to one dimension. An interactive slider demo (in the optional lab) lets you rotate a candidate z-axis and watch the projected points on the right compress or spread out in real time — confirming that the principal component is the rotation angle that maximizes that spread.

The projection formula (dot product)

Once PCA has found a direction for the z-axis, that direction is represented as a **length-1 (unit) vector**. In the worked example, the chosen z-axis direction is approximately [0.71, 0.71] (more precisely 0.707..., i.e. $1/\sqrt{2}$ in each coordinate). For a training example at $x_1 = 2$, $x_2 = 3$, its coordinate on the z-axis is the **dot product** of the point's vector with this unit vector:

MATH

```
z=[2,3].[0.71,0.71]
z=(2*0.71)+(3*0.71)
z=3.55# projected coordinate
```

So the single number 3.55 is used to represent the original two-number point (2, 3) once it is projected onto this particular z-axis.

Additional axes are always perpendicular

If a second axis (Z_2, the “second principal component”) is chosen, it is always constrained to be at 90 degrees (perpendicular) to Z_1. A third axis, if chosen, must be perpendicular to both Z_1 and Z_2, and so on for any further axes — e.g. with 50 original features, finding 3 principal components still means axis 2 $\perp$ axis 1, and axis 3 $\perp$ both.

PCA vs. Linear Regression — a common point of confusion

The instructor explicitly flags this comparison because the two techniques can look superficially similar (both involve “fitting a line” to 2D data), but they are fundamentally different algorithms with different purposes.

Linear Regression	PCA
Supervised — uses labeled data (x, y)	Unsupervised — no label y, just x_1, x_2, ...
One special variable, y, is predicted from the rest	All features are treated equally — no feature is “special”
Minimizes *vertical* distances (along the y-axis) between the fitted line and each true y	Chooses the axis that maximizes the *spread of projections* (equivalently, keeps the perpendicular projection distances small)
Goal: predict a target output y	Goal: reduce the number of features / dimensions, e.g. to visualize data

■ WATCH OUT

In 2D these two techniques can produce lines that look visually similar, tempting you to think of PCA as “regression without a label.” With more than 2 features (the typical real case), the two algorithms diverge sharply and are used for entirely different purposes — don't reach for PCA when you actually want to predict a target y, and don't reach for linear regression when you just want to reduce dimensionality.

Reconstruction: going back (approximately) from Z

Given only the projected value Z = 3.55 for the earlier example, can the original (x_1, x_2) be recovered? Not exactly — information was lost in the projection — but it can be **approximated** using the reconstruction formula: multiply Z by the same unit-length direction vector used for the projection.

MATH

```
reconstruction=Z*[0.71,0.71]
reconstruction=3.55*[0.71,0.71]
reconstruction=[2.52,2.52]# approx. of [2, 3]
```

The reconstructed point [2.52, 2.52] is close to, but not identical to, the original [2, 3] — a reasonable approximation from just one number. The distance between the original point and this reconstructed point is exactly the perpendicular line segment used during projection.

■ WATCH OUT

Reconstruction from a *reduced* number of components is always an approximation, not an exact recovery, whenever you actually threw information away (e.g. 2 features → 1 component). The one exception, shown in Part 3, is when you don't reduce dimensionality at all (e.g. 2 features → 2 components) — then reconstruction is exact.

Part 3 — Implementing PCA in scikit-learn

The standard workflow

1. **Feature scaling (conditional).** If features span very different ranges (the instructor's example: a country's GDP could be in the trillions, while other features are under 100), scale the features first so PCA can find good axes. This step is skipped if ranges are already comparable.
2. **Fit PCA.** Finds the new axes Z_1, Z_2 (and Z_3 if requested) that are the principal components. Ask for 2 or 3 components if the goal is visualization; more components are possible but harder to visualize.
3. **Check explained_variance_ratio_.** Reveals how much of the original dataset's variance/information each new axis retains — this tells you whether projecting onto the chosen axes is actually capturing most of the useful signal.
4. **Transform.** Projects each training example onto the new axes, producing 2 or 3 numbers per example that can then be plotted.

IMPORTANT scikit-learn DETAIL

`fit()` automatically performs mean normalization internally (it subtracts each feature's mean for you). You do **not** need to separately mean-normalize before calling it — that preprocessing step from Part 2 is handled inside scikit-learn's PCA implementation.

Worked code example — reducing 2 features to 1

Dataset: 6 training examples, each with 2 features (x_1, x_2). To reduce to a single number per example:

PYTHON

```python
from sklearn.decomposition import PCA

pca_1=PCA(n_components=1)
pca_1.fit(X)

pca_1.explained_variance_ratio_
# -> 0.992

pca_1.transform(X)
# first example (1, 1) -> 1.383
```

The 0.992 means the single chosen axis (Z_1) captures **99.2%** of the variability/information in the original 2-feature dataset — keeping just one number per example loses very little. `transform(X)` then returns 6 numbers, one per example; the point (1, 1) lands at 1.383, matching the optional lab's visualization where all six points collapse onto a single line (the chosen z-axis), with (1,1) landing a distance of 1.38 from the origin along it.

Worked code example — keeping 2 features as 2 components

Same dataset, but now asking for 2 principal components (2D → 2D, so no dimensionality is actually lost — this example exists mainly to build intuition about how the code and the explained-variance numbers behave):

PYTHON

```python
pca_2=PCA(n_components=2)
pca_2.fit(X)

pca_2.explained_variance_ratio_
# -> [0.992, 0.008]

pca_2.transform(X)
# first example (1, 1) -> (1.38, 0.29)
```

Z_1 still explains 99.2% of the variance and Z_2 explains the remaining 0.8% — together they sum to exactly 1 (100%), because going from 2 original dimensions to 2 new axes loses nothing. Because this is a lossless 2D → 2D transform, reconstructing back from (Z_1, Z_2) recovers the original point **exactly** — unlike the earlier 2D → 1D case, where reconstruction was only an approximation.

Practical advice: when PCA is (and isn't) worth using today

The instructor closes with candid, experience-based guidance on PCA's three classic use cases, explicitly ranking how relevant each still is:

- **Visualization (still very common).** Reduce data to 2 or 3 numbers so it can be plotted and understood — this is, by his account, the use he reaches for most today.
- **Data compression (mostly historical).** E.g. compressing 50 car features down to 10 to save database storage or network transmission time. This mattered more 10–20 years ago; modern cheap storage and fast networks have made it far less necessary.
- **Speeding up supervised learning (mostly historical).** Reducing e.g. 1,000 features to 100 before training used to meaningfully speed up older algorithms like support vector machines. With modern deep learning, this usually doesn't help much — it's now more common to feed the full high-dimensional data straight into a neural network, since PCA itself carries its own computational cost.

> ■ **WATCH OUT**
>
> If you see PCA used for compression or for speeding up training in older papers or textbooks, recognize that as a legacy use case the instructor says he rarely applies anymore — don't assume it's still standard practice with modern deep learning pipelines.

Interview Prep & Real-World Application

1. What PCA is / when to use it vs. linear regression

Interview-style questions:

- "Explain PCA to someone who only knows linear regression — how are the two different, and why can't you just use one for the other's job?"
- "You have unlabeled, high-dimensional data. Would you reach for PCA or linear regression, and why?"

Core takeaway: PCA is unsupervised and treats all features symmetrically while maximizing variance along new orthogonal axes; linear regression is supervised and minimizes vertical error against a specific target y. Never confuse "fitting a line" in PCA with "fitting a line" in regression — the objective being optimized is different.

Real-world use: A data science team exploring a brand-new, unlabeled dataset (e.g. customer behavior logs with dozens of engineered features) reaches for PCA first to visualize and sanity-check the data, long before any predictive (supervised) model is built.

2. Preprocessing requirements (mean normalization & feature scaling)

Interview-style questions:

- “Why does PCA require mean-normalized data? What goes wrong if you skip it?”
- “When would you feature-scale before PCA, and when might you skip that step?”

Core takeaway: PCA needs zero-mean features to correctly find directions of maximum variance around the origin; if features are on very different scales (e.g. GDP in trillions vs. a 0–10 index), the largest-magnitude feature will dominate the principal components unless everything is scaled comparably first. scikit-learn's fit() handles mean normalization automatically, but NOT feature scaling — that's still on you.

Real-world use: Before running PCA on a sensor dataset with mixed units (temperature in Celsius, pressure in Pascals, voltage in millivolts), an engineer standardizes each column so no single high-magnitude sensor artificially dominates the resulting principal components.

3. Principal components & explained variance

Interview-style questions:

- “What does it mean for one principal component to 'explain 92% of the variance'? What would you conclude if the top 2 components only explained 40% combined?”
- “How would you decide how many principal components to keep for a given task?”

Core takeaway: explained_variance_ratio_ tells you how much of the original information is retained by each new axis. High cumulative variance in the first few components means dimensionality reduction is 'safe' (little information lost); low cumulative variance is a signal that reducing to just 2–3 dimensions will discard a lot of real structure in the data.

Real-world use: A data scientist checks explained_variance_ratio_ before publishing a 2D PCA plot of survey data; if the top 2 components only capture 35% of the variance, they flag in the report that the visualization is a rough sketch, not a faithful summary, of the full dataset.

4. The projection & reconstruction math (dot products, orthogonality)

Interview-style questions:

- “Walk me through, mathematically, how a data point gets mapped onto a principal component axis.”
- “Why must the second principal component be orthogonal to the first? What would go wrong if it weren't?”

Core takeaway: Projection onto a component is a dot product between the data point and that component's unit-length direction vector. Components are forced to be mutually orthogonal (perpendicular) so that each new axis captures variance the previous axes didn't already capture — without that constraint, later components could just re-describe the same variance the first one already explained.

Real-world use: Implementing PCA from scratch (e.g. for a take-home ML engineering assessment) requires computing the covariance matrix and its eigenvectors; each eigenvector IS one of these orthogonal unit-length direction vectors, and the corresponding eigenvalue is proportional to the variance explained — this is the linear-algebra machinery underneath scikit-learn's fit().

5. The scikit-learn API (fit / transform / explained_variance_ratio_)

Interview-style questions:

- "What's the difference between fit(), transform(), and fit_transform() in scikit-learn's PCA, and when would you call fit() on one dataset but transform() on another?"

Core takeaway: fit() learns the principal-component directions from a dataset (typically the training set); transform() applies that already-learned projection to (possibly different) data. This separation matters in production: you fit PCA once on training data, then reuse the exact same learned axes to transform new incoming data — refitting on every new batch would give inconsistent axes over time.

Real-world use: An ML pipeline fits PCA once on a large historical training set, saves the fitted PCA object (e.g. via pickle/joblib), and reuses transform() on every new batch of incoming production data so that dimensionality reduction stays consistent across model versions.

6. Practical applications: visualization vs. compression vs. speeding up training

Interview-style questions:

- "Would you use PCA to speed up training of a modern deep learning model? Justify your answer."
- "Give an example of when PCA-based data compression would still make sense today."

Core takeaway: Visualization is PCA's dominant modern use case. Using PCA purely to speed up training is largely outdated for deep learning (feeding raw high-dimensional data directly to a neural network usually works better, since PCA adds its own compute cost); compression is only occasionally still relevant under real storage/bandwidth constraints.

Real-world use: A team building an exploratory dashboard for a 200-feature manufacturing dataset uses PCA to produce a 2D scatterplot that immediately reveals two visually distinct clusters of machine behavior — an insight that then drives a completely separate, targeted supervised-learning investigation.

Tailored Report

Written for: a learner comfortable with Python who is building ML foundations from scratch — not a complete beginner, not yet an expert.

Overview & where this fits in the specialization

These three videos are an optional detour after the recommender-systems material, introducing PCA as a second unsupervised learning algorithm (the first typically being clustering, covered earlier). PCA is presented purely as a visualization/exploration tool rather than as part of a predictive pipeline, which is why it's placed as optional, standalone content rather than woven into the core supervised-learning track. Conceptually, it leans on ideas already built earlier in the specialization: feature scaling/normalization (from linear regression), and the general notion of "fitting" a model to data via `fit()` in scikit-learn (the same pattern used for regression and classification models earlier in the course).

Foundational vs. supplementary

Foundational — internalize	Supplementary / context
• What a principal component is (max-variance axis) • Why mean normalization / feature scaling matters before PCA • The projection formula (dot product with a unit vector) • explained_variance_ratio_ and what it tells you • The fit() / transform() workflow in scikit-learn • PCA vs. linear regression — the conceptual distinction	• The country-GDP and 3D-pancake examples (intuition-builders, not required to recall in detail) • Historical uses of PCA for compression / speeding up SVM-era training • Exact numeric values from the worked examples (1.383, 3.55, etc.) — useful for following along, not for memorizing

Commonly tricky parts of this lecture

- **PCA vs. linear regression.** Because both involve “fitting a line” to 2D data, it's easy to blur the two mentally. The instructor calls this out explicitly and repeatedly because the objective functions (vertical residuals vs. maximized projected variance) and the presence/absence of a label y are genuinely different — this is the single point most worth re-deriving by hand.
- **“Maximizing variance” vs. “minimizing projection distance.”** The instructor mentions these are two ways of describing the same optimal axis without fully proving the equivalence on screen. It's worth sitting with why a large spread of projections corresponds to small perpendicular distances (hint: think Pythagorean theorem — the original distance from the origin to a point is fixed, and it splits into a projected-distance component and a perpendicular-distance component).
- **Reconstruction is only exact when no dimensions are actually dropped.** Learners sometimes assume reconstruction always approximately recovers the original data equally well — the 2D→1D example (approximate) vs. 2D→2D example (exact) in Part 3 exists specifically to make this distinction concrete.

Next steps to solidify this material

1. **Re-derive by hand:** take a small 2–3 point 2D dataset, mean-normalize it, pick two or three candidate unit-vector directions, and manually compute the projected variance for each to confirm which one PCA would actually choose.
2. **Implement:** reproduce the scikit-learn code walkthrough yourself (n_components=1 and n_components=2 on the same toy dataset) and verify your explained_variance_ratio_ and transform() outputs match the lecture's numbers before moving to a real dataset.
3. **Re-watch / dig deeper:** if the “maximize variance = minimize projection distance” equivalence didn't fully click, look up how PCA connects to eigenvectors/eigenvalues of the covariance matrix — the lecture deliberately stays at the intuitive/API level and doesn't cover this derivation, but it's the natural next layer down and it's the layer interviewers sometimes probe.

Takeaway:

PCA is an unsupervised technique that finds new, mutually orthogonal axes maximizing the variance captured when data is projected onto them. It requires mean-normalized (and often scaled) input, is fundamentally different from linear regression (no label, all features treated equally), and its scikit-learn workflow is fit() → explained_variance_ratio_ → transform(). Today it's used mainly for visualization — compressing high-dimensional data to 2 or 3 numbers per example so it can actually be plotted and understood.

Study guide generated from lecture transcript — Andrew Ng Machine Learning Specialization, PCA (optional videos).